

OpenCTR 机器人控制器 源码阅读指南

固件：OpenCTR_H60V36_R750ROS V3.61（塔克创新 XTARK）

主控：STM32F4（标准外设库 StdPeriph）+ FreeRTOS

用途：ROS 机器人移动底盘控制器（支持 6 种底盘运动学）

本指南按照 4 条主线帮你读懂整个工程：

1. 目录结构

—— 代码是怎么分层组织的
2. 任务架构与主逻辑

—— 程序从上电到运行的执行流
3. 函数调用关系 / 数据流

—— 一条速度指令是怎么变成电机 PWM 的
4. 接口与参数要求

—— 每个模块对外暴露的函数、参数含义、通信协议

一、项目概览

项目	说明
芯片	STM32F4 系列（Cortex-M7 端口用于 F7，此处用 F4 端口）
系统	FreeRTOS 实时操作系统（抢占式多任务）
库	STM32F4xx 标准外设库 + CMSIS + USB Host Library
版本	ROBOT_FW_VER = "V3.61"（定义于 Robot/ax_robot.h）
当前编译底盘	ROBOT_TYPE = ROBOT_FWD（四轮差速），见 ax_robot.h:159
控制上位机	ROS（通过 USB 串口 / TTL 串口 / CAN 三选一通信）

支持的 6 种底盘类型（编译期由 ROBOT_TYPE 宏切换）：

宏	值	底盘	主动轮
ROBOT_MEC	0x01	麦克纳姆轮	A/B/C/D 四轮
ROBOT_FWD	0x02	四轮差速	A/B/C/D 四轮
ROBOT_AKM	0x03	阿克曼转向	A/B 两轮 + 舵机
ROBOT_TWD	0x04	两轮差速	A/B 两轮
ROBOT_TNK	0x05	履带坦克	A/B 两轮
ROBOT_OMT	0x06	三轮全向	A/B/C 三轮

二、目录结构（先看目录）

工程分为 4 层，你只需要关注最上面两层（Robot/ 和 Driver/），下面两层是芯片厂商和开源库，一般不用改。

OpenCTR_H60V36_R750ROS_V3.61.0602/		
— Robot/	★★★	应用层（机器人主逻辑，最先读这里）
— Driver/	★★	驱动层（硬件抽象 HAL，按需查阅）
— FreeRTOS/		实时操作系统内核（一般不动）
— Libraries/		STM32 标准外设库 + CMSIS（一般不动）
— Project/		Keil 工程文件

2.1 应用层 Robot/ —— 机器人大脑（重点）

文件	职责	关键函数
main.c	程序入口：外设初始化 + 创建 FreeRTOS 任务	main(), Start_Task()
ax_robot.c/.h	主任务与全局定义：核心循环、IMU 处理、数据回传、按键/显示/灯效/琐事任务	Robot_Task(), ROBOT_SendDataToRos()
ax_control.c/.h	控制源解析：手柄 / APP / 航模遥控器 → 目标速度	AX_CTL_Gpd/App/RemoteSbus()
ax_kinematics.c/.h	运动学解析：目标速度 ↔ 各轮速度（6 种底盘各一份实现）	AX_ROBOT_Kinematics(), AX_ROBOT_Stop()
ax_speed.c/.h	电机 PID 闭环：轮子目标速度 → PWM	AX_SPEED_PidCtlA/B/C/D()
ax_akm.c/.h	阿克曼专用：矢量速度 → 前轮转角	AX_AKM_WToAngle()
ax_light.c/.h	RGB 灯带 6 种灯效	AX_LIGHT_Show()
stm32f4xx_it.c	中断向量入口（多数具体中断在各驱动里实现）	—
system_stm32f4xx.c	时钟系统初始化	SystemInit()
FreeRTOSConfig.h	RTOS 配置	—

2.2 驱动层 Driver/ —— 硬件抽象（按需查）

文件	外设	一句话
ax_motor	电机	4 路直流电机 PWM 调速
ax_encoder	编码器	4 路正交编码器计数（定时器）
ax_imu	IMU	QMI8658 六轴陀螺仪/加速度计（软件 I2C）
ax_servo	舵机	6 路 PWM 舵机（阿克曼转向用 S6）
ax_uart1	串口1	调试串口（printf 输出）

文件	外设	一句话
ax_uart2	串口2	ROS USB 通信 (X-Protocol)
ax_uart3	串口3	蓝牙 APP 通信
ax_uart4	串口4	ROS TTL 通信 (X-Protocol)
ax_can	CAN	ROS CAN 通信 (1Mbps)
ax_sbus	SBUS	航模遥控器接收
ax_gamepad	USB 手柄	USB Host HID 无线手柄
ax_rgb	WS2812	8 颗 RGB 灯带
ax_oled	OLED	128×64 屏显示 (在 Driver/ax_oled/)
ax_vin	ADC	电池电压检测
ax_key	按键	单按键扫描
ax_led	LED	红/绿指示灯 (宏定义)
ax_beep	蜂鸣器	鸣叫 (宏定义)
ax_eeprom	EEPROM	参数掉电保存 (I2C)
ax_delay	延时	软件微秒/毫秒延时
ax_sys	系统	位带操作等底层宏

三、任务架构与主逻辑（主逻辑怎么看）

3.1 启动流程

阅读顺序：main.c → main() → Start_Task() → Robot_Task()



- └ 设置舵机零偏角
- └ 陀螺仪零点校准（采 10 次求平均，存入 ax_imu_gyro_offset[]）
- └ OLED 显示主界面
- └ 创建 5 个常驻任务（临界区内）：
 - | Robot / Key / Light / Disp / Trivia
- └ vTaskDelete(自身) ← 建完就自杀，释放栈

3.2 任务清单 (FreeRTOS)

任务	优先级	栈	周期	职责
Start_Task	1	128	一次性	参数加载、陀螺仪校准、创建其余任务后删除自己
Robot_Task	3	256	20ms (50Hz)	核心控制环 ：控制源解析→运动学→PID→发数据
Key_Task	4	128	50ms	按键扫描（短按切灯效 / 中按阿克曼校准 / 长按恢复出厂）
Light_Task	5	128	30ms (≈33Hz)	RGB 灯效刷新
Disp_Task	6	128	100ms	OLED 刷新（模式/电压/陀螺仪/轮速）
Trivia_Task	10	128	500ms	琐事：电池管理、IMU 校准触发、灯效存储、蜂鸣、绿灯心跳

优先级数字越大越高（FreeRTOS 惯例）。Robot_Task 用 vTaskDelayUntil 保证**严格 50Hz**定频，这与运动学中的 PID_RATE=50 一一对应。

3.3 核心控制环 Robot_Task (50Hz, 最重要的一段)

位于 ax_robot.c:92，每 20ms 执行一次：

```
while(1) {
    vTaskDelayUntil(&PreviousWakeTime, 20ms);    // 定频 50Hz

    // 1) 控制源解析（仅非 ROS 模式；ROS 模式速度由中断直接写入）
    if (ax_control_mode == CTL_GPD) AX_CTL_Gpd();           // 手柄
    else if (ax_control_mode == CTL_APP) AX_CTL_App();      // 蓝牙APP
    else if (ax_control_mode == CTL_RMS) AX_CTL_RemoteSbus(); // 航模

    // 2) 运动控制
    if (ax_robot_move_enable) AX_ROBOT_Kinematics();        // 运动学+PID+电机
    else AX_ROBOT_Stop();                                   // 全部停车

    // 3) 读 IMU 并做坐标变换
    ROBOT_IMUHandle();

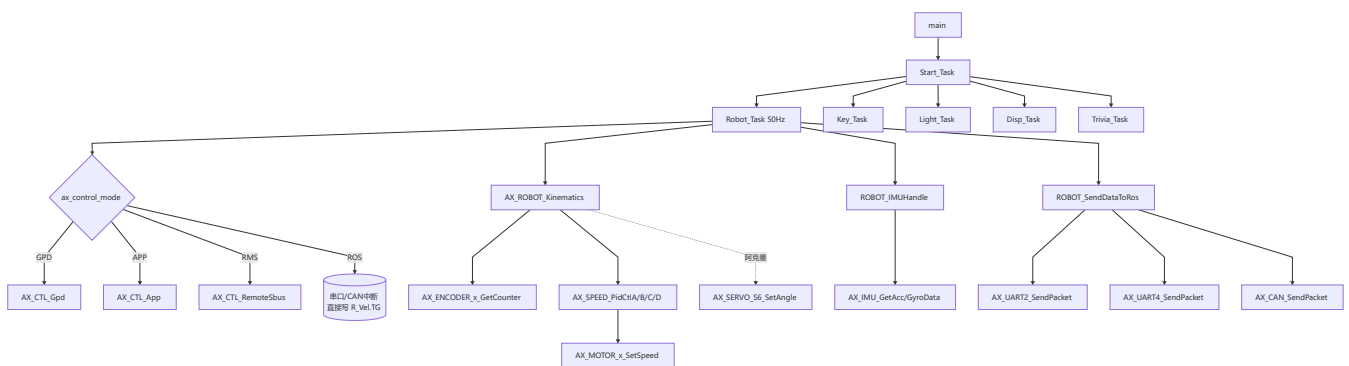
    // 4) 回传数据给 ROS（串口 + CAN）
    ROBOT_SendDataToRos();
}
```

```
}
```

关键点：ROS 模式（默认 `CTL_ROS=0`）下，目标速度**不在这个循环里解析**，而是在串口/CAN 中断里直接写入 `R_Vel.TG_*`。循环只负责"消费"这些目标值。

四、函数调用关系（调用图）

4.1 整体调用关系



4.2 三个层次的职责边界

应用层 Robot/ "怎么动"
控制源解析 → 运动学解析 → PID 闭环
`R_Vel`(速度) ↔ `R_Wheel`(轮速) ↔ PWM

驱动层 Driver/ "怎么控硬件"
Set电机PWM / 读编码器 / 读IMU / 收发串口CAN / 舵机 / 灯

库层 Libraries + FreeRTOS "寄存器 & 调度"

五、核心数据流：一条速度指令的完整旅程 ★

这是理解整个工程的**主脉络**。以默认 ROS 控制、四轮差速为例：

5.1 下行：ROS → 电机（控制指令）

① ROS 上位机通过 USB 串口发送 X-Protocol 帧：

AA 55 | 0B | 50 | xhi xlo yhi ylo whi wlo | 校验
└─帧码 ID_URX_VEL=0x50

② `USART2_IRQHandler()` (`ax_uart2.c:109`) 逐字节收帧、校验

↓ 校验通过

```
ax_control_mode = CTL_ROS;           // 自动切到 ROS 模式
R_Vel.TG_IX = (xhi<<8)|xlo;          // 目标 X 速度 (mm/s)
R_Vel.TG_IY = (yhi<<8)|ylo;          // 目标 Y 速度 (mm/s)
R_Vel.TG_IW = (whi<<8)|wlo;          // 目标角速度 (mrad/s)
```

```

③ Robot_Task (50Hz) → AX_ROBOT_Kinematics() (ax_kinematics.c)
└ 目标速度限幅 (±R_VX/VY/VW_LIMIT)
└ 整型→浮点: TG_FX = TG_IX/1000.0 (转成 m/s)
└ 【逆运动学】机器人速度 → 4 个轮子目标速度 R_wheel_x.TG
└ 四差速: A=Fx - Fw·(轮距·系数/2) B=Fx + Fw·(...) C/D 同理
└ 【PID】AX_SPEED_PidCtlA(TG, RT) → PWM (增量式 PD)
└ AX_MOTOR_x_SetSpeed(±PWM) // 输出到电机

```

5.2 上行：编码器 → ROS (里程反馈)

```

① 每 20ms, AX_ROBOT_Kinematics() 开头先读编码器:
R_wheel_A.RT = 编码器计数 × WHEEL_SCALE // 实时轮速 m/s
AX_ENCODER_A_SetCounter(0); // 清零, 下周期重新累计

② 【正运动学】4 个轮速 → 机器人实时速度:
R_Vel.RT_IX = (A+B+C+D)/4 × 1000 // m/s → mm/s
R_Vel.RT_IW = (-A+B-C+D)/(2·系数·轮距)×1000

③ ROBOT_IMUHandle(): 读 IMU 并做 IMU→ROS 坐标变换
R_Imu.ACC_X = imu[Y]; ACC_Y = -imu[X]; ACC_Z = imu[Z]; // 轴向映射

④ ROBOT_SendDataToRos(): 打包 20 字节, 帧码 0x10, 三路同时发:
AX_UART2_SendPacket() / AX_UART4_SendPacket() / AX_CAN_SendPacket()
内容 = [加速度XYZ | 陀螺仪XYZ | 实时速度XYW | 电池电压]

```

闭环关系：编码器测得 **RT** (实际轮速) → 与逆运动学算出的 **TG** (目标轮速) 做差 → PID 输出 PWM → 驱动电机使 **RT** 逼近 **TG**。

六、运动学解析 (ax_kinematics.c)

同一个函数名 `AX_ROBOT_Kinematics()` 用 `#if (ROBOT_TYPE==xxx)` **条件编译**, 6 种底盘各一份。每份都遵循相同的 5 步套路:

1. **读编码器** → 实时轮速 `R_wheel_x.RT` (并清零计数器)
2. **正运动学**: 轮速 → 机器人实际速度 `R_Vel.RT_I*` (用于回传)
3. **目标速度限幅** + 整型转浮点 `R_Vel.TG_F*`
4. **逆运动学**: 机器人目标速度 → 轮子目标速度 `R_wheel_x.TG`
5. **PID + 输出**: `AX_SPEED_PidCtl*( )` → `AX_MOTOR_x_SetSpeed( )`

各底盘逆运动学核心公式 ($k = \text{轮距}/2 + \text{轴距}/2$):

底盘	逆解公式要点
麦轮 MEC	$A=F_x-F_y-F_w \cdot k$, $B=F_x+F_y+F_w \cdot k$, $C=F_x+F_y-F_w \cdot k$, $D=F_x-F_y+F_w \cdot k$
四差速 FWD	$A=C=F_x-F_w \cdot k$, $B=D=F_x+F_w \cdot k$ (k 含 <code>FWD_WB_SCALE</code> 轮距系数)
阿克曼 AKM	由转弯半径 $R=\text{轴距}/\tan(\text{转角})$ 计算左右后轮差速 + 舵机转角

底盘	逆解公式要点
两差速 TWD	$A = Fx - Fw \cdot \text{轮距} / 2$, $B = Fx + Fw \cdot \text{轮距} / 2$
履带 TNK	同两差速
三全向 OMT	120° 三轮矢量分解

阿克曼比较特殊：只有 A/B 后轮驱动，前轮靠 **S6 舵机** 转向；`AX_AKM_WToAngle()` (`ax_akm.c`) 把 ROS 的 (vx, vw) 矢量速度换算成前轮转角，并做**最小转弯半径限制** `AKM_TURN_R_MINI=0.35m`。

七、通信协议（接口重点）★

7.1 X-Protocol 串口协议（变帧长）

用于 UART2 (USB) / UART3 (蓝牙) / UART4 (TTL) 。

帧结构：	AA 55		LEN		ID		DATA...		SUM
	_____		_____		_____		_____		_____
	双帧头		帧长		帧码		数据(高位在前)		校验和

- **帧头**：固定 `0xAA 0x55`（双帧头抗干扰）
- **帧长 LEN**：整帧总字节数 = 数据长度 + 5
- **帧码 ID**：标识帧用途（见下表）
- **数据**：高字节在前（大端），可含 8/16/32 位数据
- **校验和 SUM**：前面所有字节累加和的低 8 位

7.2 帧码定义 (`ax_robot.h:204`)

下行 (ROS/上位机 → 控制器)：

帧码	宏	含义	数据
0x50	<code>ID_URX_VEL</code>	速度控制	X(2) Y(2) W(2), int16, 单位×1000
0x51	<code>ID_URX_IMU</code>	陀螺仪零偏校准	1 字节标志
0x52	<code>ID_URX_LG</code>	灯光控制	M,S,T,R,G,B (6 字节)
0x53	<code>ID_URX_LS</code>	灯光效果保存到 EEPROM	—
0x54	<code>ID_URX_BP</code>	蜂鸣器控制	1=响 0=停
0x5A	<code>ID_URX_RTY</code>	底盘型号校验	1 字节型号，与本机比对

上行 (控制器 → ROS)：

帧码	宏	含义	数据 (20 字节)
0x10	ID_UTX_DATA	综合数据回传	加速度XYZ(6) + 陀螺仪XYZ(6) + 速度XYW(6) + 电压(2)

7.3 CAN 协议 (ax_can.c , 1Mbps)

帧 ID	宏	方向	内容
0x0181	ID_CANRX_VEL	收	X(2) Y(2) W(2) 预留(2)
0x0121	ID_CANTX_MSG1	发	加速度 XYZ
0x0122	ID_CANTX_MSG2	发	陀螺仪 XYZ
0x0123	ID_CANTX_MSG3	发	速度 XYW + 电压

7.4 蓝牙 APP 协议 (ax_control.c + ax_robot.h:215)

帧码	含义
0x31 ID_BLERX_YG	摇杆模式运动
0x32 ID_BLERX_SB	手柄模式运动
0x33 ID_BLERX_ZL	重力模式运动
0x41 ID_BLERX_LG	灯光控制
0x42 ID_BLERX_LS	灯光保存

7.5 速度物理量与单位约定

量	变量	单位 (整型 I)	限幅
X 线速度	R_Vel.*_IX	mm/s (m/s×1000)	±1500 (R_VX_LIMIT)
Y 线速度	R_Vel.*_IY	mm/s	±1200 (R_VY_LIMIT)
角速度	R_Vel.*_IW	mrad/s (rad/s×1000)	±6280 (R_VW_LIMIT)

_I* = 整型 (收发用) , _F* = 浮点 (运算用, = 整型/1000) ; TG_ = 目标, RT_ = 实时。

八、接口 / API 参考（各模块函数与参数）

8.1 应用层核心接口

函数	参数	返回	说明
<code>AX_ROBOT_Kinematics()</code>	无	无	一次完整运动学+PID+输出 (50Hz 调用)
<code>AX_ROBOT_Stop()</code>	无	无	4 电机 PWM 置 0
<code>AX_SPEED_PidCtlA/B/C/D(target, current)</code>	<code>float</code> 目标/实际轮速 (m/s)	<code>int16_t</code> PWM	增量式 PID (实为 PD)，累加到 <code>motor_pwm_out[]</code>
<code>AX_AKM_WToAngle(vx, vw)</code>	<code>int16_t</code> 速度/角速度	<code>int16_t</code> 转角 ×1000(rad)	阿克曼矢量→前轮转角
<code>AX_CTL_Gpd/App/RemoteSbus()</code>	无	无	各控制源→ <code>R_Vel.TG_*</code>
<code>AX_LIGHT_Show()</code>	无	无	按 <code>R_Light.M</code> 显示对应灯效

PID 参数：`ax_motor_kp=800`, `ax_motor_kd=1000` (`ax_robot.c`)。算法：
`pwm += kp·bias + kd·(bias - bias_last)`, `bias = target - current`。

8.2 驱动层接口速查

电机 `ax_motor.h`

函数	参数	说明
<code>AX_MOTOR_AB_Init()</code> / <code>AX_MOTOR_CD_Init()</code>	无	初始化 A/B、C/D PWM
<code>AX_MOTOR_x_SetSpeed(speed)</code>	<code>int16_t</code> PWM, 负值反转	x = A/B/C/D

编码器 `ax_encoder.h`

函数	参数/返回	说明
<code>AX_ENCODER_x_Init()</code>	无	x = A/B/C/D
<code>AX_ENCODER_x_GetCounter()</code>	返回 <code>uint16_t</code> (需转 <code>int16</code> 判正负)	读周期内计数
<code>AX_ENCODER_x_SetCounter(count)</code>	<code>uint16_t</code> (一般传 0)	清零/设值

IMU `ax_imu.h` (QMI8658)

函数	参数	说明
<code>AX_IMU_Init()</code>	无	初始化

函数	参数	说明
AX_IMU_GetAccData(pbuf)	int16_t[3] 出参	加速度 XYZ 原始值
AX_IMU_GetGyroData(pbuf)	int16_t[3] 出参	陀螺仪 XYZ 原始值
AX_IMU_ConfigAcc(range, odr, mode)	枚举	量程/速率/滤波
AX_IMU_ConfigGyro(range, odr, mode)	枚举	同上

舵机 ax_servo.h （阿克曼用 S6）

函数	参数	说明
AX_SERVO_S6_Init()	无	初始化
AX_SERVO_S6_SetAngle(angle)	int16_t 角度 (×10, 含零偏)	设转角
AX_SERVO_S6_Release()	无	释放 (掉力矩)
(另有 S1~S5 同款接口)		扩展舵机

串口 ax_uart2/3/4.h

函数	参数	说明
AX_UARTx_Init(baud)	uint32_t 波特率	U2/U4=230400, U3=115200, U1=230400
AX_UARTx_SendPacket(pbuf, len, num)	数据指针 / 长度(≤32) / 帧码	X-Protocol 发送
AX_UART2/3_GetData(pbuf)	出参缓冲	返回是否有新数据

CAN ax_can.h

函数	参数	说明
AX_CAN_Init()	无	1Mbps, 收全部 ID
AX_CAN_SendPacket(id, len, pbuf)	ID / 长度(≤8) / 指针	发标准帧, 返回是否成功

RGB ax_rgb.h （8 颗 WS2812）

函数	参数	说明
AX_RGB_Init()	无	初始化
AX_RGB_SetFullColor(b, r, g)	3× uint8_t	全部同色（注意形参顺序 B,R,G）
AX_RGB_SetPixelColor(pixel[8][3])	逐灯颜色数组	可变数组

SBUS `ax_sbus.h` / **手柄** `ax_gamepad.h`

函数	参数	说明
<code>AX_SBUS_GetRxData(pbuf)</code>	<code>uint16_t[8]</code> 出参	8 通道值 (中位≈992)
<code>GAMEPAD_Decode(...)</code>	USB 回调	解析后填充全局 <code>GamePad</code>

`GamePad` (`GAMEPAD_Type_t`) : `K1/K2` 按键位、`LX/LY/RX/RY` 摇杆(-128~127)、`LT/RT` 扳机(0~255)。

其他小驱动

模块	关键接口
电压 <code>ax_vin</code>	<code>AX_VIN_GetVol_X100()</code> → 电压×100 (如 1119=11.19V)
按键 <code>ax_key</code>	<code>AX_KEY_Scan()</code> → 按下非 0
LED <code>ax_led</code>	宏: <code>AX_LED_Red_On/Off/Toggle()</code> 、 <code>AX_LED_Green_*</code>
蜂鸣器 <code>ax_beep</code>	宏: <code>AX_BEEP_On/Off/Toggle()</code>
EEPROM <code>ax_eeprom</code>	<code>AX_EEPROM_ReadOneByte(addr)</code> / <code>WriteOneByte(addr,data)</code> / <code>Read/Write(addr,buf,num)</code>

九、关键全局变量 (`ax_robot.c` 定义, `ax_robot.h` 声明)

变量	类型	含义
<code>R_Vel</code>	<code>ROBOT_Velocity</code>	机器人速度 (目标/实时, 整型/浮点, X/Y/W)
<code>R_wheel_A/B/C/D</code>	<code>ROBOT_wheel</code>	每轮的实时速度 RT / 目标速度 TG / 输出 PWM
<code>R_Imu</code>	<code>ROBOT_Imu</code>	IMU 加速度+陀螺仪 (已转 ROS 坐标)
<code>R_Light</code>	<code>ROBOT_Light</code>	灯效模式 M/S/T + 颜色 RGB
<code>R_Bat_Vol</code>	<code>uint16_t</code>	电池电压×100
<code>ax_control_mode</code>	<code>uint8_t</code>	控制源: 0=ROS / 1=手柄 / 2=APP / 3=航模
<code>ax_robot_move_enable</code>	<code>uint8_t</code>	运动使能 (0=急停)
<code>ax_motor_kp / kd</code>	<code>int16_t</code>	PID 参数
<code>ax_imu_gyro_offset[3]</code>	<code>int16_t</code>	陀螺仪零偏校准值
<code>ax_akm_angle / ax_akm_offset</code>	<code>int16_t</code>	阿克曼前轮转角 / 舵机零偏

变量	类型	含义
<code>ax_light_save_flag</code> / <code>ax_beep_ring</code> / <code>ax_imu_calibrate_flag</code>	—	任务间通信的标志位

EEPROM 参数地图：

地址	内容
0x00	初始化标记（0x88=已初始化）
0x20~0x25	灯效 M,S,T,R,G,B
0x50	阿克曼舵机零偏

十、建议的阅读路径

1. **先跑通主脉络：** `main.c` → `Start_Task` → `Robot_Task` 的 4 步循环。
2. **理解一条指令：** 顺着第五章的"速度指令旅程"，从 `USART2_IRQHandler` 追到 `AX_MOTOR_A_SetSpeed`。
3. **抠运动学：** 只看你的底盘对应的那个 `#if (ROBOT_TYPE==...)` 分支（当前是 `ROBOT_FWD`）。
4. **看闭环：** `ax_speed.c` 的增量式 PID，配合编码器反馈理解调速。
5. **查协议：** 第七章通信协议表，对照 ROS 端驱动收发。
6. **按需查驱动：** 需要改硬件行为时再翻 `Driver/` 对应模块。

改机常见入口：

- 换底盘类型 → `ax_robot.h:159` 的 `ROBOT_TYPE`
- 调 PID → `ax_robot.c` 的 `ax_motor_kp` / `ax_motor_kd`
- 调机械参数（轮径/轮距/减速比）→ `ax_robot.h` 的 `xxx_WHEEL_*` 宏
- 调速度上限 → `ax_robot.h` 的 `R_VX/VY/VW_LIMIT`
- 改通信波特率 → `main.c` 的 `AX_UARTx_Init()`

本指南基于 OpenCTR V3.61 源码整理，供快速上手阅读参考。